



BigQuery Lab Guide — GCP Free Tier

Table of Contents

- **Module 1 — BigQuery Datasets**
 - Lab 1.1 — Creating Datasets
 - Lab 1.2 — Public Datasets
 - Lab 1.3 — Dataset Properties
 - Lab 1.4 — Create and Query Clustered Tables
 - Lab 1.5 — Create and Query External Tables
- **Module 2 — BigQuery Tables**
 - Lab 2.1 — Create and Use Tables
 - Lab 2.2 — Table Schemas
 - Lab 2.3 — Create, Manage, and Query Partitioned Tables
- **Module 3 — BigQuery Analyze**
 - Lab 3.1 — Introduction to BigQuery Analysis
 - Lab 3.2 — Run a Query
 - Lab 3.3 — Write Query Results
 - Lab 3.4 — GoogleSQL ANSI Standard
 - Lab 3.5 — Querying with Arrays
 - Lab 3.6 — Querying JSON Data
 - Lab 3.7 — Multi-Statement Queries
 - Lab 3.8 — Recursive CTEs
 - Lab 3.9 — Creating and Running Saved Queries
 - Lab 3.10 — Optimize Queries
 - Lab 3.11 — Query External Tables
 - Lab 3.12 — Logical Views
 - Lab 3.13 — Materialized Views



Setup

Step 1 — Access BigQuery Sandbox

1. Go to console.cloud.google.com
2. Sign in with your Google account
3. In the top search bar, type **BigQuery** and click the service
4. If prompted to create a project, click **Create Project** → name it **bigquery-lab** → click **Create**
5. BigQuery Sandbox activates automatically — no billing required

🔗 **Sandbox vs Free Tier** The Sandbox requires no credit card but tables expire in 60 days and DML is disabled. The Free Tier requires billing info but tables are permanent and all features work — you still pay nothing within the 1 TB / 10 GB monthly limits.

Step 2 — Open BigQuery Studio

Once in the BigQuery console, you'll see:

- **Explorer panel** (left) — lists your projects, datasets, and tables
 - **Editor panel** (center) — where you write SQL
 - **Results panel** (bottom) — query output, schema, job info
-

Module 1 – BigQuery Datasets

What is a Dataset?

A **dataset** in BigQuery is a top-level container that groups related tables and views. It is analogous to a **schema** or **database** in traditional SQL systems. Every table must live inside a dataset. A dataset belongs to a single GCP project and lives in a specific geographic region.

Hierarchy: `project_id.dataset_id.table_id` Example: `myproject.sales_data.orders`

Lab 1.1 – Creating Datasets

Concepts

Term	Definition
Dataset ID	Unique name within a project. Can contain letters, numbers, underscores. Max 1024 chars. Case-sensitive.
Location	Geographic region where data is stored (e.g., <code>US</code> , <code>EU</code> , <code>asia-south1</code>). Cannot be changed after creation.
Default Table Expiration	Auto-deletes all tables in the dataset after N milliseconds. Useful for temporary datasets.
Labels	Key-value metadata tags (e.g., <code>env:prod</code> , <code>team:analytics</code>) used for cost allocation and filtering.

Method 1 – Console (UI)

1. In the **Explorer** panel, click the three-dot menu next to your project name
2. Click **Create dataset**
3. Fill in:
 - **Dataset ID:** `ecommerce`
 - **Location type:** `Multi-region` → `US`
 - Leave other options as default
4. Click **Create dataset**

Method 2 – SQL (DDL)

```
-- Create a basic dataset
-- Note: DDL for datasets uses CREATE SCHEMA in GoogleSQL
CREATE SCHEMA IF NOT EXISTS `bigquery-lab.ecommerce`
OPTIONS (
  location = 'US',
  description = 'E-commerce data for BigQuery labs',
  labels = [("env", "lab"), ("team", "data-engineering")]
);
```

SQL

```
-- Create a dataset with a default table expiration of 90 days
-- 7776000000 milliseconds = 90 days
CREATE SCHEMA IF NOT EXISTS `bigquery-lab.temp_workspace`
OPTIONS (
  location = 'US',
  default_table_expiration_days = 90,
  description = 'Temporary workspace - tables auto-expire after 90 days'
);
```

SQL

Method 3 – bq CLI

```
# Basic dataset creation
bq mk --dataset \
  --location=US \
  --description="E-commerce lab dataset" \
  bigquery-lab:ecommerce

# With table expiration (in seconds)
bq mk --dataset \
  --location=US \
  --default_table_expiration=7776000 \
  bigquery-lab:temp_workspace

# Verify dataset was created
bq ls bigquery-lab:
```

SHELL

Verify

```
-- List all datasets in your project
SELECT
  schema_name AS dataset_id,
  creation_time,
  location
FROM `bigquery-lab`.INFORMATION_SCHEMA.SCHEMATA
ORDER BY creation_time DESC;
```

SQL

✓ Expected Output You should see `ecommerce` listed with your project, creation timestamp, and `US` as location.

Lab 1.2 – Public Datasets

Concepts

 **What are Public Datasets?** BigQuery hosts hundreds of free, publicly available datasets through the Google Cloud Public Dataset Program. These live in the `bigquery-public-data` project, which is accessible to all GCP users. Querying them counts against your 1 TB monthly free quota.

Popular public datasets include:

Dataset	Description
<code>bigquery-public-data.new_york_citibike</code>	NYC Citi Bike trips since 2013
<code>bigquery-public-data.chicago_crime</code>	Chicago crime reports
<code>bigquery-public-data.samples.shakespeare</code>	All works of Shakespeare tokenized
<code>bigquery-public-data.census_bureau_usa</code>	US Census population data
<code>bigquery-public-data.github_repos</code>	3TB+ of open source code
<code>bigquery-public-data.noaa_gsod</code>	Global weather data since 1929

Step 1 – Pin a Public Dataset

1. In the Explorer panel, click + Add
2. Click **Public datasets**
3. Search for **NYC Citi Bike Trips**
4. Click **View dataset** → it appears pinned in your Explorer

Step 2 – Explore the Schema

```
-- Inspect table schema without running a full query
-- Use backtick quoting for project.dataset.table paths
SELECT
  column_name,
  data_type,
  is_nullable
FROM `bigquery-public-data`.`new_york_citibike`.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'citibike_trips'
ORDER BY ordinal_position;
```

SQL

Step 3 – Query Public Data

```
-- How many trips were taken each year?
-- Always preview bytes scanned before running on large tables
SELECT
  EXTRACT(YEAR FROM starttime) AS year,
  COUNT(*) AS total_trips,
  ROUND(AVG(tripduration) / 60, 1) AS avg_duration_minutes
FROM `bigquery-public-data.new_york_citibike.citibike_trips`
WHERE starttime IS NOT NULL
GROUP BY year
ORDER BY year;
```

SQL

```
-- Top 10 most popular start stations
SELECT
  start_station_name,
  COUNT(*) AS trip_count
FROM `bigquery-public-data.new_york_citibike.citibike_trips`
GROUP BY start_station_name
ORDER BY trip_count DESC
LIMIT 10;
```

SQL

⚠ Watch Your Bytes! Before running a query, look at the top-right of the query editor — it shows estimated bytes to be scanned. The citibike dataset is ~4 GB. Always use `LIMIT` during exploration.

Step 4 – Use Shakespeare Dataset (Small, Safe)

```
-- Count word frequencies across all Shakespeare works
SELECT
  word,
  SUM(word_count) AS total_occurrences
FROM `bigquery-public-data.samples.shakespeare`
GROUP BY word
ORDER BY total_occurrences DESC
LIMIT 20;
```

SQL

```
-- Which plays use the word "love" most?
SELECT
  corpus AS play,
  SUM(word_count) AS love_count
FROM `bigquery-public-data.samples.shakespeare`
WHERE LOWER(word) = 'love'
GROUP BY corpus
ORDER BY love_count DESC
LIMIT 10;
```

SQL

Lab 1.3 – Dataset Properties

Concepts

Property	Description
Access Controls	IAM roles (Viewer, Editor, Owner) applied at dataset level override project-level permissions
Default Encryption	Google-managed by default. Can use Customer-Managed Encryption Keys (CMEK).
Default Collation	How string comparisons are performed. und:ci enables case-insensitive matching.
Max Time Travel	How far back in time you can query historical data. Default 7 days, max 7 days on free tier.

View Current Properties

```
-- View dataset metadata
SELECT
  *
FROM `bigquery-lab`.INFORMATION_SCHEMA.SCHEMATA_OPTIONS
WHERE schema_name = 'ecommerce';
```

SQL

Alter Dataset Properties (DDL)

```
-- Update description and add labels to existing dataset
ALTER SCHEMA `bigquery-lab.ecommerce`
SET OPTIONS (
  description = 'Production e-commerce dataset - orders, customers, products',
  labels = [("env", "lab"), ("team", "analytics"), ("cost-center", "engineering")]
);
```

SQL

```
-- Set default table expiration on existing dataset (7 days = 604800 seconds)
ALTER SCHEMA `bigquery-lab.ecommerce`
SET OPTIONS (
  default_table_expiration_days = 7
);
```

SQL

Manage Dataset Access

```
-- Grant read access to a specific user (requires billing account, not sandbox)
-- This is for reference - run in a billed project
GRANT `roles/bigquery.dataViewer`
ON SCHEMA `bigquery-lab.ecommerce`
TO 'user:analyst@example.com';
```

SQL

```
# Using bq CLI - show dataset info
bq show --format=prettyjson bigquery-lab:ecommerce

# Update description
bq update --description "Updated description" bigquery-lab:ecommerce
```

SHELL

Lab 1.4 – Create and Query Clustered Tables

Concepts

 What is Clustering?

Clustering organizes data within a table's storage blocks based on the values of one or more columns. When you query a clustered table with a filter on a clustered column, BigQuery skips blocks that don't match — this is called **block pruning**.

Unlike partitioning (which divides a table into segments), clustering **sorts data within** those segments.

Aspect	Detail
Max clustered columns	4
Supported types	STRING, INT64, NUMERIC, DATE, TIMESTAMP, BOOL, GEOGRAPHY
Best for	High-cardinality columns you filter on frequently
Cost benefit	Reduces bytes scanned → saves free tier quota
Auto-reclustering	BigQuery re-clusters in the background as data is added

Step 1 – Create Sample Data

SQL

```
-- First, create the base dataset table we'll use across labs
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_raw` (
  order_id      STRING      NOT NULL,
  customer_id   STRING      NOT NULL,
  product_id    STRING      NOT NULL,
  category      STRING,
  status        STRING,
  amount        NUMERIC,
  created_at    TIMESTAMP
);

-- Insert sample data
INSERT INTO `bigquery-lab.ecommerce.orders_raw` VALUES
  ('ORD-001', 'CUST-101', 'PROD-A1', 'Electronics', 'completed', 299.99, '2024-01-15 10:30:00 UTC'),
  ('ORD-002', 'CUST-102', 'PROD-B2', 'Clothing', 'completed', 49.50, '2024-01-15 11:00:00 UTC'),
  ('ORD-003', 'CUST-101', 'PROD-C3', 'Electronics', 'pending', 150.00, '2024-02-10 09:15:00 UTC'),
  ('ORD-004', 'CUST-103', 'PROD-A1', 'Electronics', 'cancelled', 299.99, '2024-02-20 14:45:00 UTC'),
  ('ORD-005', 'CUST-104', 'PROD-D4', 'Home & Garden', 'completed', 75.25, '2024-03-05 16:20:00 UTC'),
  ('ORD-006', 'CUST-102', 'PROD-E5', 'Clothing', 'completed', 89.99, '2024-03-12 08:00:00 UTC'),
  ('ORD-007', 'CUST-105', 'PROD-A1', 'Electronics', 'pending', 299.99, '2024-04-01 12:00:00 UTC'),
  ('ORD-008', 'CUST-103', 'PROD-F6', 'Books', 'completed', 14.99, '2024-04-10 17:30:00 UTC');
```

 **Sandbox Note** `INSERT` requires a billing account. In the Sandbox, use `CREATE TABLE AS SELECT` instead (shown below).

SQL

```
-- Sandbox-friendly alternative: use CREATE TABLE AS SELECT
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_raw` AS
SELECT * FROM UNNEST([
  STRUCT('ORD-001' AS order_id, 'CUST-101' AS customer_id, 'PROD-A1' AS product_id,
    'Electronics' AS category, 'completed' AS status, NUMERIC '299.99' AS amount,
    TIMESTAMP '2024-01-15 10:30:00 UTC' AS created_at),
  STRUCT('ORD-002', 'CUST-102', 'PROD-B2', 'Clothing', 'completed', NUMERIC '49.50', TIMESTAMP '2024-01-15 11:00:00
UTC'),
  STRUCT('ORD-003', 'CUST-101', 'PROD-C3', 'Electronics', 'pending', NUMERIC '150.00', TIMESTAMP '2024-02-10 09:15:00
UTC'),
  STRUCT('ORD-004', 'CUST-103', 'PROD-A1', 'Electronics', 'cancelled', NUMERIC '299.99', TIMESTAMP '2024-02-20 14:45:00
UTC'),
  STRUCT('ORD-005', 'CUST-104', 'PROD-D4', 'Home & Garden', 'completed', NUMERIC '75.25', TIMESTAMP '2024-03-05 16:20:00
UTC'),
  STRUCT('ORD-006', 'CUST-102', 'PROD-E5', 'Clothing', 'completed', NUMERIC '89.99', TIMESTAMP '2024-03-12 08:00:00
UTC'),
  STRUCT('ORD-007', 'CUST-105', 'PROD-A1', 'Electronics', 'pending', NUMERIC '299.99', TIMESTAMP '2024-04-01 12:00:00
UTC'),
  STRUCT('ORD-008', 'CUST-103', 'PROD-F6', 'Books', 'completed', NUMERIC '14.99', TIMESTAMP '2024-04-10 17:30:00
UTC')
]);
```

Step 2 – Create a Clustered Table

SQL

```
-- Create a clustered table from the raw data
-- Cluster by category first (most frequently filtered), then status
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_clustered`
CLUSTER BY category, status -- up to 4 columns; order matters for pruning
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

```
-- Create a table clustered on a single high-cardinality column
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_by_customer`
CLUSTER BY customer_id
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

Step 3 – Query and Compare

SQL

```
-- Query with cluster filter – BigQuery prunes non-matching blocks
SELECT
  order_id,
  amount,
  created_at
FROM `bigquery-lab.ecommerce.orders_clustered`
WHERE category = 'Electronics'
AND status = 'completed';
```

SQL

```
-- Combine clustering with aggregation
SELECT
  category,
  status,
  COUNT(*)      AS order_count,
  SUM(amount)   AS total_revenue,
  AVG(amount)   AS avg_order_value
FROM `bigquery-lab.ecommerce.orders_clustered`
WHERE category IN ('Electronics', 'Clothing')
GROUP BY category, status
ORDER BY category, total_revenue DESC;
```

Step 4 — Inspect Clustering Info

SQL

```
-- Check if a table is clustered and on which columns
SELECT
  table_name,
  clustering_ordinal_position,
  column_name
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.COLUMNS
WHERE clustering_ordinal_position IS NOT NULL
ORDER BY table_name, clustering_ordinal_position;
```

🔗 When to Use Clustering

- You frequently filter on the same 1–4 columns
- The table is larger than 1 GB (smaller tables see no benefit)
- Those columns have many distinct values (high cardinality like `customer_id`, `product_id`, `category`)

Lab 1.5 — Create and Query External Tables

Concepts

📖 **What is an External Table?** An external table (also called a federated table) allows BigQuery to query data that lives outside BigQuery storage — typically in Google Cloud Storage (GCS), Google Drive, Cloud Bigtable, or Cloud Spanner. The data is never imported; BigQuery reads it on demand.

Feature	External Table	Native Table
Data location	GCS, Drive, etc.	BigQuery managed storage
Query cost	Full scan every time (no cache)	Benefits from clustering/partitioning
Load required	No	Yes
DML support	No	Yes
Clustering	Not supported	Supported
Speed	Slower	Faster

Step 1 — Prepare Data in Cloud Storage

🔗 For this lab we'll use a public GCS bucket. In real usage, you'd upload your own CSV/JSON/Parquet files to your own bucket.

SHELL

```
# Create a GCS bucket (replace YOUR_PROJECT with your project ID)
gsutil mb -l US gs://bigquery-lab-external-YOUR_PROJECT

# Create a sample CSV file
cat > /tmp/products.csv << 'EOF'
product_id,name,category,price,stock_quantity
PROD-A1,Wireless Headphones,Electronics,199.99,45
PROD-B2,Cotton T-Shirt,Clothing,24.99,200
PROD-C3,Smart Watch,Electronics,349.99,12
PROD-D4,Garden Hose,Home & Garden,39.99,80
PROD-E5,Running Shoes,Clothing,89.99,55
PROD-F6,Python Cookbook,Books,34.99,120
EOF

# Upload to GCS
gsutil cp /tmp/products.csv gs://bigquery-lab-external-YOUR_PROJECT/products/products.csv
```

Step 2 – Create the External Table

SQL

```
-- Create external table pointing to GCS CSV
-- Replace YOUR_BUCKET with your actual bucket name
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.products_external`
(
  product_id STRING,
  name STRING,
  category STRING,
  price NUMERIC,
  stock_quantity INT64
)
OPTIONS (
  format = 'CSV',
  uris = ['gs://YOUR_BUCKET/products/*.csv'],
  skip_leading_rows = 1, -- skip header row
  description = 'Product catalog - external, sourced from GCS'
);
```

SQL

```
-- External table from JSON files
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.events_external`
OPTIONS (
  format = 'NEWLINE_DELIMITED_JSON',
  uris = ['gs://YOUR_BUCKET/events/*.json']
);
```

SQL

```
-- External table from Parquet (most efficient format for BQ)
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.logs_parquet`
OPTIONS (
  format = 'PARQUET',
  uris = ['gs://YOUR_BUCKET/logs/*.parquet']
);
```

Step 3 – Query the External Table

SQL

```
-- Basic query on external table
SELECT *
FROM `bigquery-lab.ecommerce.products_external`
WHERE category = 'Electronics'
ORDER BY price DESC;
```

SQL

```
-- Join external table with native table
-- Every run re-reads from GCS - no caching
SELECT
  o.order_id,
  o.customer_id,
  o.amount,
  p.name      AS product_name,
  p.category,
  p.price     AS list_price,
  o.amount - p.price AS discount_applied
FROM `bigquery-lab.ecommerce.orders_clustered` AS o
JOIN `bigquery-lab.ecommerce.products_external` AS p
  ON o.product_id = p.product_id
ORDER BY o.created_at;
```

⚠ **Cost Warning** External table queries are not cached. Running the same query twice bills you twice. For frequently-used external data, consider loading it into a native table:

SQL

```
-- Materialize external table into native storage
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.products_native`
AS
SELECT * FROM `bigquery-lab.ecommerce.products_external`;
```

Module 2 – BigQuery Tables

📖 **BigQuery Table Types** BigQuery supports multiple table types:

- **Standard (Native) table** — data stored in BigQuery's Capacitor columnar format
- **External table** — data in GCS/Drive (covered in Lab 1.5)
- **View** — saved SQL query, no stored data (Lab 3.12)
- **Materialized view** — precomputed cached query results (Lab 3.13)
- **Partitioned table** — table divided into segments for performance (Lab 2.3)
- **Clustered table** — data sorted within blocks (Lab 1.4)

Lab 2.1 – Create and Use Tables

Concepts

Term	Definition
CREATE TABLE	DDL to define a new table with an explicit schema
CREATE TABLE AS SELECT (CTAS)	Creates a table and populates it from a query result in one step
CREATE OR REPLACE	Drops and recreates if the table exists; atomic operation
CREATE IF NOT EXISTS	Only creates if the table doesn't exist; safe for idempotent scripts
Temporary table	Session-scoped table (prefixed with <code>_SESSION_</code>) that auto-deletes when session ends

Method 1 – Empty Table with Explicit Schema

SQL

```
-- Create a customers table with full schema
CREATE TABLE IF NOT EXISTS `bigquery-lab.ecommerce.customers` (
  customer_id STRING NOT NULL,
  email STRING,
  first_name STRING,
  last_name STRING,
  country STRING,
  signup_date DATE,
  is_premium BOOL DEFAULT FALSE,
  metadata JSON, -- flexible semi-structured field
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP()
)
OPTIONS (
  description = 'Customer master table',
  require_partition_filter = FALSE
);
```

Method 2 – CTAS (Create Table As Select)

SQL

```
-- Create a summary table from another table
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.category_summary`
OPTIONS (
  description = 'Aggregated revenue by category',
  labels = [("type", "summary")]
)
AS
SELECT
  category,
  COUNT(DISTINCT customer_id) AS unique_customers,
  COUNT(*) AS total_orders,
  SUM(amount) AS total_revenue,
  ROUND(AVG(amount), 2) AS avg_order_value,
  MIN(created_at) AS first_order_at,
  MAX(created_at) AS last_order_at
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY category;
```

Method 3 – Temporary Tables

SQL

```
-- Temporary tables exist only for the current session
-- Great for multi-step analysis without permanent storage
CREATE TEMP TABLE top_customers AS
SELECT
  customer_id,
  COUNT(*) AS order_count,
  SUM(amount) AS lifetime_value
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY customer_id
HAVING order_count >= 2;

-- Use the temp table in subsequent queries (same session)
SELECT
  t.customer_id,
  t.lifetime_value,
  t.order_count
FROM top_customers AS t
ORDER BY lifetime_value DESC;
```

Copy, Rename, and Delete Tables

```
-- Copy a table (snapshot)
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_backup`
AS SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

```
-- Drop a table
DROP TABLE IF EXISTS `bigquery-lab.ecommerce.orders_backup`;
```

SQL

```
# Using bq CLI
bq cp bigquery-lab:ecommerce.orders_raw bigquery-lab:ecommerce.orders_backup

# Copy across datasets or projects
bq cp \
  bigquery-lab:ecommerce.orders_raw \
  another-project:another_dataset.orders_copy

# Delete table
bq rm -f bigquery-lab:ecommerce.orders_backup
```

SHELL

DML Operations (Requires Billing Account)

```
-- INSERT individual rows
INSERT INTO `bigquery-lab.ecommerce.customers`
(customer_id, email, first_name, last_name, country, signup_date, is_premium)
VALUES
('CUST-101', 'alice@example.com', 'Alice', 'Wong', 'IN', '2023-06-01', TRUE),
('CUST-102', 'bob@example.com', 'Bob', 'Sharma', 'IN', '2023-08-15', FALSE),
('CUST-103', 'carlos@example.com', 'Carlos', 'Mendes', 'BR', '2024-01-10', FALSE),
('CUST-104', 'diana@example.com', 'Diana', 'Patel', 'IN', '2024-02-20', TRUE),
('CUST-105', 'elena@example.com', 'Elena', 'Kowalski', 'PL', '2024-03-05', FALSE);

-- UPDATE rows (requires billing account)
UPDATE `bigquery-lab.ecommerce.customers`
SET is_premium = TRUE
WHERE customer_id = 'CUST-102';

-- DELETE rows (requires billing account)
DELETE FROM `bigquery-lab.ecommerce.customers`
WHERE country = 'PL';

-- MERGE (upsert) – update if exists, insert if not
MERGE `bigquery-lab.ecommerce.customers` AS target
USING (
  SELECT 'CUST-106' AS customer_id, 'frank@example.com' AS email,
         'Frank' AS first_name, 'Lima' AS last_name, 'MX' AS country,
         DATE '2024-05-01' AS signup_date, FALSE AS is_premium
) AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
  UPDATE SET email = source.email, is_premium = source.is_premium
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, first_name, last_name, country, signup_date, is_premium)
VALUES (source.customer_id, source.email, source.first_name, source.last_name,
        source.country, source.signup_date, source.is_premium);
```

SQL

Lab 2.2 – Table Schemas

Concepts

 **Schema Design in BigQuery** A schema defines the structure of a table: column names, data types, modes (nullable/required/repeated), and descriptions. BigQuery's type system is richer than most databases, including native support for **ARRAY**, **STRUCT**, **JSON**, and **GEOGRAPHY**.

Data Types Reference

```

STRING          -- Variable length Unicode string
BYTES           -- Variable length binary data
INT64           -- 64-bit integer (-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807)
FLOAT64         -- Double-precision floating point (use for scientific data)
NUMERIC         -- Exact decimal (29 digits, 9 decimal places) - use for money
BIGNUMERIC      -- Exact decimal (76 digits, 38 decimal places)
BOOL            -- TRUE or FALSE
DATE            -- 'YYYY-MM-DD' (no timezone)
TIME            -- 'HH:MM:SS[.ffffff]' (no timezone)
DATETIME        -- 'YYYY-MM-DD HH:MM:SS' (no timezone)
TIMESTAMP       -- Absolute point in time with UTC timezone
INTERVAL        -- Duration (years-months-days hours:min:sec.micro)
ARRAY<T>        -- Ordered list of values of type T
STRUCT<...>     -- Record with named fields of different types
JSON            -- Semi-structured JSON document
GEOGRAPHY       -- Geographic point/shape (WGS84)
RANGE<T>        -- A contiguous range of DATE, DATETIME, or TIMESTAMP

```

Column Modes

Mode	Meaning	NULL allowed?
NULLABLE (default)	Column can contain NULL	Yes
REQUIRED	Column must have a value	No
REPEATED	Column is an array (ARRAY)	No — but can be empty

Create Table with Rich Schema

SQL

```
-- A table showcasing all important schema features
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_detailed` (
  -- Primary identifier
  order_id      STRING      NOT NULL OPTIONS (description = 'Unique order identifier'),

  -- Timestamps
  created_at    TIMESTAMP   NOT NULL,
  updated_at    TIMESTAMP,

  -- Customer info (nested STRUCT)
  customer      STRUCT<
    id          STRING,
    email       STRING,
    country_code STRING,
    is_premium  BOOL
  >
  OPTIONS (description = 'Embedded customer snapshot at time of order'),

  -- Line items (REPEATED STRUCT = array of records)
  line_items    ARRAY<STRUCT<
    product_id  STRING,
    product_name STRING,
    quantity    INT64,
    unit_price  NUMERIC,
    discount_pct NUMERIC
  >>
  OPTIONS (description = 'One entry per product in the order'),

  -- Payment
  payment_method STRING,
  total_amount   NUMERIC   NOT NULL,
  currency       STRING    DEFAULT 'USD',

  -- Flexible metadata
  tags           ARRAY<STRING>,    -- e.g., ['flash-sale', 'mobile-app']
  attributes     JSON,            -- any extra key-value data

  -- Geo
  shipping_location GEOGRAPHY OPTIONS (description = 'Delivery coordinates')
)
OPTIONS (
  description = 'Orders with full nested schema - showcases BQ type system'
);
```

Schema Auto-Detection

SQL

```
-- When loading from GCS, BigQuery can auto-detect schema
-- (shown as bq CLI since it's most common use case)
-- bq load \
--   --autodetect \
--   --source_format=CSV \
--   bigquery-lab:ecommerce.auto_table \
--   gs://YOUR_BUCKET/data.csv
```

Modify Schema (Add Columns)

```
-- Add new columns to existing table (backward compatible)
ALTER TABLE `bigquery-lab.ecommerce.customers`
ADD COLUMN IF NOT EXISTS phone_number STRING,
ADD COLUMN IF NOT EXISTS loyalty_tier STRING,
ADD COLUMN IF NOT EXISTS last_order_at TIMESTAMP;
```

SQL

```
-- Add column with description
ALTER TABLE `bigquery-lab.ecommerce.customers`
ADD COLUMN IF NOT EXISTS referral_source STRING
  OPTIONS (description = 'How the customer found us: organic, paid, referral, social');
```

SQL

```
-- Drop a column (irreversible!)
ALTER TABLE `bigquery-lab.ecommerce.customers`
DROP COLUMN IF EXISTS phone_number;
```

SQL

Inspect Schema Programmatically

```
-- Full column info for a table
SELECT
  table_name,
  column_name,
  ordinal_position,
  data_type,
  is_nullable,
  column_default,
  description
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'customers'
ORDER BY ordinal_position;
```

SQL

```
# bq CLI - show table schema as JSON
bq show --schema --format=prettyjson \
  bigquery-lab:ecommerce.customers
```

SHELL

Lab 2.3 — Create, Manage, and Query Partitioned Tables

Concepts

 **What is Table Partitioning?** Partitioning divides a table into segments called partitions based on a column's value. When you query with a filter on the partition column, BigQuery only reads the matching partitions — dramatically reducing data scanned and cost.

Partition Types

Type	Column Type	Description
Ingestion-time	Auto (<code>_PARTITIONTIME</code>)	Partitioned by when data was loaded
Date/Timestamp	DATE, TIMESTAMP, DATETIME	Partitioned by values in a date column
Integer range	INT64	Partitioned by numeric ranges

Partition Granularity (Date/Timestamp)

Granularity	Number of partitions	Best for
<code>HOUR</code>	Up to 10,000	High-frequency event data
<code>DAY</code> (default)	Up to 10,000	Most analytical workloads
<code>MONTH</code>	Up to 10,000	Long historical data

Granularity	Number of partitions	Best for
YEAR	Up to 10,000	Multi-decade data

Step 1 – Create a Date-Partitioned Table

```
-- Partitioned by day on the created_at column
-- Each day's data is stored separately
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_partitioned`
PARTITION BY DATE(created_at)    -- partition column must be DATE, TIMESTAMP, or DATETIME
OPTIONS (
  description          = 'Orders partitioned by creation date',
  partition_expiration_days = 365,    -- partitions older than 1 year auto-delete
  require_partition_filter = TRUE    -- forces callers to include a WHERE on the partition column
)
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

```
-- Partition by month (useful when you have years of data)
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_monthly`
PARTITION BY DATE_TRUNC(DATE(created_at), MONTH)
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

Step 2 – Create an Integer Range Partitioned Table

```
-- Partition customer_id ranges: 0-999, 1000-1999, etc.
-- Useful for numeric IDs, user cohorts, zip codes
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_by_id_range`
PARTITION BY RANGE_BUCKET(
  CAST(REGEXP_EXTRACT(customer_id, r'\d+') AS INT64),
  GENERATE_ARRAY(100, 200, 10) -- start=100, end=200, interval=10 → partitions: [100,110), [110,120), ...
)
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

Step 3 – Create a Partitioned + Clustered Table (Best of Both)

```
-- Partitioned by date AND clustered by category and status
-- This is the recommended pattern for large analytical tables
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_optimized`
PARTITION BY DATE(created_at)
CLUSTER BY category, status
OPTIONS (
  description = 'Orders - partitioned by date, clustered by category+status'
)
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

Step 4 – Query Partitioned Tables

```
-- Good query: uses partition filter → only reads Jan 2024 data
SELECT
  order_id,
  customer_id,
  amount
FROM `bigquery-lab.ecommerce.orders_partitioned`
WHERE DATE(created_at) = '2024-01-15';    -- partition pruning!
```

SQL

SQL

```
-- Query a date range
SELECT
  DATE(created_at) AS order_date,
  COUNT(*) AS orders,
  SUM(amount) AS daily_revenue
FROM `bigquery-lab.ecommerce.orders_partitioned`
WHERE DATE(created_at) BETWEEN '2024-01-01' AND '2024-03-31' -- Q1 2024 only
GROUP BY order_date
ORDER BY order_date;
```

SQL

```
-- This query FAILS if require_partition_filter = TRUE (by design)
-- SELECT * FROM `bigquery-lab.ecommerce.orders_partitioned`;
-- Error: "Queries over table must have at least one filter..."

-- Workaround: use _PARTITIONDATE pseudo-column to select specific partitions
SELECT *
FROM `bigquery-lab.ecommerce.orders_partitioned`
WHERE _PARTITIONDATE = '2024-01-15';
```

Step 5 – Manage Partitions

SQL

```
-- List all partitions in a partitioned table
SELECT
  partition_id,
  total_rows,
  total_logical_bytes,
  last_modified_time
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.PARTITIONS
WHERE table_name = 'orders_partitioned'
ORDER BY partition_id;
```

SHELL

```
# bq CLI – list partitions
bq show --format=prettyjson bigquery-lab:ecommerce.orders_partitioned

# Copy a specific partition to another table
bq cp \
  'bigquery-lab:ecommerce.orders_partitioned$20240115' \
  bigquery-lab:ecommerce.orders_jan15_snapshot
```

Module 3 – BigQuery Analyze

Lab 3.1 – Introduction to BigQuery Analysis

Concepts

 **BigQuery as an Analytics Engine** BigQuery is a serverless, columnar data warehouse. It uses a distributed query engine called Dremel that processes queries across thousands of nodes in parallel. Key characteristics:

- **Columnar storage** — only reads columns referenced in your SELECT
- **Massively parallel** — automatically scales to your query size
- **Serverless** — no infrastructure to manage; pay per query
- **Separation of storage and compute** — stored data is cheap; compute happens only when you query

How BigQuery Executes a Query

```

Your SQL
↓
Query Parser (validates syntax, resolves table refs)
↓
Query Optimizer (chooses execution plan, estimates cost)
↓
Execution Plan (stages, workers, shuffle operations)
↓
Distributed Workers (read only needed columns/partitions)
↓
Results (returned to client or written to table)

```

Key Concepts for Efficient Analysis

Concept	Description
Bytes processed	Total data scanned — what you pay for
Bytes billed	Minimum 10 MB per query; rounded up
Slot	Unit of BigQuery compute (CPU + RAM). Free tier = shared slots
Job	Every query is a job with an ID, start time, duration, and bytes processed
Result cache	Identical queries on unchanged data return cached results for free within 24h

Check Estimated Cost Before Running

```

-- Set maximum bytes billed to 1 GB - query fails if it would scan more
-- Set this in: More → Query Settings → Maximum bytes billed

-- Or use the bq CLI:
-- bq query --maximum_bytes_billed=1073741824 "SELECT ..."

```

SQL

Query Execution Details

After running any query, click **Job information** tab to see:

- Bytes processed
- Elapsed time
- Bytes shuffled
- Slot time used

Lab 3.2 – Run a Query

Concepts

Term	Meaning
Query editor	The SQL input area in BigQuery Studio
Dry run	Estimates bytes to be scanned without actually running the query
Keyboard shortcut	Ctrl+Enter (Windows/Linux) or Cmd+Enter (Mac) to run

Basic SELECT Patterns

SQL

```
-- Select specific columns (always prefer over SELECT *)
SELECT
  order_id,
  customer_id,
  amount,
  created_at
FROM `bigquery-lab.ecommerce.orders_raw`
LIMIT 10;
```

SQL

```
-- Filter with WHERE
SELECT
  order_id,
  amount,
  status
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed'
  AND amount > 100.00
  AND DATE(created_at) >= '2024-02-01';
```

SQL

```
-- Aggregate functions
SELECT
  status,
  COUNT(*)           AS order_count,
  SUM(amount)        AS total_revenue,
  ROUND(AVG(amount), 2) AS avg_order,
  MIN(amount)        AS min_order,
  MAX(amount)        AS max_order
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY status
ORDER BY total_revenue DESC;
```

SQL

```
-- JOINS
SELECT
  o.order_id,
  c.first_name,
  c.last_name,
  c.country,
  o.amount,
  o.status
FROM `bigquery-lab.ecommerce.orders_raw` AS o
LEFT JOIN `bigquery-lab.ecommerce.customers` AS c
  ON o.customer_id = c.customer_id
WHERE o.status = 'completed'
ORDER BY o.created_at DESC;
```

Window Functions

```
-- Running total of revenue per category over time
SELECT
  DATE(created_at) AS order_date,
  category,
  amount,
  SUM(amount) OVER (
    PARTITION BY category
    ORDER BY DATE(created_at)
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS running_revenue,
  RANK() OVER (PARTITION BY category ORDER BY amount DESC) AS rank_in_category
FROM `bigquery-lab.ecommerce.orders_raw`
ORDER BY category, order_date;
```

SQL

```
-- LAG and LEAD: compare with previous/next row
SELECT
  order_id,
  customer_id,
  amount,
  created_at,
  LAG(amount) OVER (PARTITION BY customer_id ORDER BY created_at) AS prev_order_amount,
  LEAD(amount) OVER (PARTITION BY customer_id ORDER BY created_at) AS next_order_amount,
  amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY created_at) AS change_from_last
FROM `bigquery-lab.ecommerce.orders_raw`
ORDER BY customer_id, created_at;
```

SQL

Dry Run (Estimate Cost Without Executing)

```
# bq CLI dry run - shows bytes to be scanned without running
bq query --dry_run \
  "SELECT * FROM `bigquery-lab.ecommerce.orders_raw`"
```

SHELL

Lab 3.3 – Write Query Results

Concepts

 **Destination Tables** By default, query results are returned to the console (max 10 GB in browser, larger via API). You can instead write results to a destination table for:

- Sharing results with others
- Using results in downstream queries
- Persisting intermediate computation

Method 1 – Via Console

1. In the query editor, click **More** → **Query settings**
2. Under **Destination**, check **Set a destination table for query results**
3. Enter: Dataset = `ecommerce`, Table = `orders_completed_2024`
4. Choose **Write preference**: Overwrite / Append / Write if empty
5. Run the query

Method 2 — CTAS

SQL

```
-- Write query results to a new permanent table
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_completed_2024`
AS
SELECT
  order_id,
  customer_id,
  amount,
  DATE(created_at) AS order_date
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed'
AND EXTRACT(YEAR FROM created_at) = 2024;
```

Method 3 — Export to Cloud Storage

SQL

```
-- EXPORT DATA to GCS in various formats
-- Note: requires billing account
EXPORT DATA OPTIONS (
  uri          = 'gs://YOUR_BUCKET/exports/orders_*.csv', -- * for sharding
  format       = 'CSV',
  header       = TRUE,
  overwrite    = TRUE
)
AS
SELECT
  order_id,
  customer_id,
  amount,
  created_at
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed';
```

SQL

```
-- Export as Parquet (most efficient for large datasets)
EXPORT DATA OPTIONS (
  uri    = 'gs://YOUR_BUCKET/exports/orders_*.parquet',
  format = 'PARQUET'
)
AS
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;
```

SHELL

```
# bq CLI extract
bq extract \
  --destination_format CSV \
  --print_header=true \
  bigquery-lab:ecommerce.orders_raw \
  gs://YOUR_BUCKET/exports/orders_*.csv
```

Lab 3.4 — GoogleSQL ANSI Standard

Concepts

- 📖 **What is GoogleSQL?** GoogleSQL (formerly "Standard SQL" in BigQuery) is BigQuery's primary SQL dialect. It is largely ANSI SQL-2011 compliant with extensions for BigQuery-specific features (arrays, structs, JSON, geography, ML).

Always use GoogleSQL — the legacy SQL dialect is deprecated.

Standard Clauses

SQL

```
-- Full SELECT statement structure
SELECT
  -- 3. Column expressions evaluated here
  column1,
  FUNCTION(column2)           AS alias,
  CASE WHEN condition THEN a ELSE b END AS conditional_col

FROM table1                  -- 1. Source table

-- 2. Filter rows before grouping
WHERE column1 = 'value'
  AND column2 > 100
  AND column3 IN ('a', 'b', 'c')
  AND column4 BETWEEN 10 AND 50
  AND column5 LIKE '%pattern%'
  AND column6 IS NOT NULL

-- 4. Group by for aggregation
GROUP BY column1, alias

-- 5. Filter groups after aggregation
HAVING COUNT(*) > 5

-- 6. Order results
ORDER BY alias DESC, column1 ASC

-- 7. Limit results
LIMIT 100

-- 8. Offset (skip first N rows)
OFFSET 20;
```

CTEs (Common Table Expressions)

SQL

```
-- CTEs using WITH clause - named subqueries for readability
WITH

-- Step 1: revenue by customer
customer_revenue AS (
  SELECT
    customer_id,
    SUM(amount) AS total_spent,
    COUNT(*) AS order_count
  FROM `bigquery-lab.ecommerce.orders_raw`
  WHERE status = 'completed'
  GROUP BY customer_id
),

-- Step 2: classify customers into tiers
customer_tiers AS (
  SELECT
    customer_id,
    total_spent,
    order_count,
    CASE
      WHEN total_spent >= 500 THEN 'Platinum'
      WHEN total_spent >= 200 THEN 'Gold'
      WHEN total_spent >= 50 THEN 'Silver'
      ELSE 'Bronze'
    END AS tier
  FROM customer_revenue
)

-- Final query uses the CTEs
SELECT
  tier,
  COUNT(*) AS customer_count,
  SUM(total_spent) AS tier_revenue,
  AVG(total_spent) AS avg_spend
FROM customer_tiers
GROUP BY tier
ORDER BY avg_spend DESC;
```

String Functions

SQL

```
SELECT
  -- Case
  UPPER('hello')           AS upper_result,    -- 'HELLO'
  LOWER('WORLD')           AS lower_result,    -- 'world'

  -- Trimming
  TRIM(' spaced ')         AS trimmed,         -- 'spaced'
  LTRIM(' left')           AS left_trimmed,    -- 'left'

  -- Substrings
  SUBSTR('BigQuery', 1, 3)  AS substr_result,   -- 'Big'
  LEFT('BigQuery', 3)       AS left_result,     -- 'Big'
  RIGHT('BigQuery', 5)     AS right_result,    -- 'Query'

  -- Search
  STRPOS('BigQuery', 'Query') AS pos,          -- 4
  CONTAINS_SUBSTR('BigQuery', 'ery') AS contains, -- TRUE

  -- Replace / Format
  REPLACE('foo-bar', '-', '_') AS replaced,    -- 'foo_bar'
  FORMAT('%s has %d items', 'cart', 3) AS formatted, -- 'cart has 3 items'
  CONCAT('Hello', ' ', 'World') AS concatenated,

  -- Regex
  REGEXP_EXTRACT('ORD-12345', r'\d+') AS extracted_num, -- '12345'
  REGEXP_REPLACE('a1b2c3', r'\d', '') AS letters_only, -- 'abc'
  REGEXP_CONTAINS('test@mail.com', r'@') AS is_email;   -- TRUE
```

Date/Time Functions

SQL

```
SELECT
  CURRENT_DATE()           AS today,
  CURRENT_TIMESTAMP()      AS now_utc,
  DATE_ADD(CURRENT_DATE(), INTERVAL 7 DAY) AS next_week,
  DATE_DIFF(DATE '2024-12-31', DATE '2024-01-01', DAY) AS days_in_year,
  DATE_TRUNC(CURRENT_DATE(), MONTH) AS first_of_month,
  EXTRACT(YEAR FROM CURRENT_TIMESTAMP()) AS current_year,
  EXTRACT(MONTH FROM CURRENT_TIMESTAMP()) AS current_month,
  EXTRACT(DOW FROM CURRENT_DATE()) AS day_of_week, -- 1=Sun, 7=Sat
  FORMAT_DATE('%B %d, %Y', CURRENT_DATE()) AS formatted_date,
  PARSE_DATE('%Y-%m-%d', '2024-06-15') AS parsed_date,
  TIMESTAMP_DIFF(
    TIMESTAMP '2024-06-01 12:00:00',
    TIMESTAMP '2024-06-01 08:30:00',
    MINUTE
  ) AS minutes_diff; -- 210
```

Lab 3.5 – Querying with Arrays

Concepts

📖 **Arrays in BigQuery** An **ARRAY** is an ordered list of zero or more values of the same type. In BigQuery, a column of type **ARRAY<T>** replaces what you'd model as a child table in a relational database. Arrays allow you to store related data without joins, reducing shuffle and storage.

Key rule: A BigQuery table cannot have an **ARRAY** at the top level of a repeated field within a **STRUCT** within an **ARRAY** (no nesting arrays inside arrays directly). But you can have **ARRAY<STRUCT<..., ARRAY<...>>>**.

Creating Tables with Arrays

SQL

```
-- Table with array columns
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.user_sessions` AS
SELECT * FROM UNNEST([
  STRUCT(
    'SESSION-001' AS session_id,
    'CUST-101' AS customer_id,
    TIMESTAMP '2024-04-01 10:00:00 UTC' AS started_at,
    ['homepage', 'electronics', 'product-A1', 'cart', 'checkout'] AS page_views,
    [STRUCT('item-1' AS item_id, 'PROD-A1' AS product_id, 299.99 AS price),
     STRUCT('item-2', 'PROD-C3', 349.99)] AS cart_items
  ),
  STRUCT(
    'SESSION-002', 'CUST-102',
    TIMESTAMP '2024-04-01 11:30:00 UTC',
    ['homepage', 'clothing', 'product-B2'],
    [STRUCT('item-1', 'PROD-B2', 49.50)]
  ),
  STRUCT(
    'SESSION-003', 'CUST-101',
    TIMESTAMP '2024-04-02 09:00:00 UTC',
    ['homepage', 'search', 'electronics', 'product-C3', 'cart'],
    [STRUCT('item-1', 'PROD-C3', 349.99), STRUCT('item-2', 'PROD-F6', 14.99)]
  )
]);
```

ARRAY Functions

SQL

```
-- ARRAY_LENGTH: count elements
SELECT
  session_id,
  ARRAY_LENGTH(page_views) AS pages_visited,
  ARRAY_LENGTH(cart_items) AS items_in_cart
FROM `bigquery-lab.ecommerce.user_sessions`;
```

SQL

```
-- Subscript: access by index (0-based with OFFSET, 1-based with ORDINAL)
SELECT
  session_id,
  page_views[OFFSET(0)] AS first_page,      -- 0-based
  page_views[ORDINAL(1)] AS also_first_page, -- 1-based (equivalent)
  page_views[SAFE_OFFSET(99)] AS safe_out_of_bounds -- returns NULL instead of error
FROM `bigquery-lab.ecommerce.user_sessions`;
```

SQL

```
-- UNNEST: flatten array into rows (one row per element)
SELECT
  session_id,
  page_view,
  ROW_NUMBER() OVER (PARTITION BY session_id ORDER BY pos) AS step_number
FROM `bigquery-lab.ecommerce.user_sessions`,
     UNNEST(page_views) AS page_view WITH OFFSET AS pos
ORDER BY session_id, pos;
```

SQL

```
-- UNNEST a STRUCT array
SELECT
  s.session_id,
  item.item_id,
  item.product_id,
  item.price
FROM `bigquery-lab.ecommerce.user_sessions` AS s,
     UNNEST(s.cart_items) AS item
ORDER BY s.session_id, item.item_id;
```

SQL

```
-- ARRAY_AGG: aggregate rows back into arrays
SELECT
  category,
  ARRAY_AGG(order_id ORDER BY created_at) AS order_history,
  ARRAY_AGG(DISTINCT status) AS seen_statuses,
  ARRAY_AGG(amount ORDER BY amount DESC LIMIT 3) AS top_3_amounts
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY category;
```

SQL

```
-- Check if a value exists in an array using IN UNNEST
SELECT
  session_id,
  customer_id,
  'cart' IN UNNEST(page_views) AS reached_cart,
  'checkout' IN UNNEST(page_views) AS completed_checkout
FROM `bigquery-lab.ecommerce.user_sessions`;
```

SQL

```
-- ARRAY construction functions
SELECT
  ARRAY_CONCAT([1, 2], [3, 4]) AS merged,           -- [1, 2, 3, 4]
  GENERATE_ARRAY(1, 5) AS one_to_five,             -- [1, 2, 3, 4, 5]
  GENERATE_ARRAY(0, 10, 2) AS evens,                -- [0, 2, 4, 6, 8, 10]
  GENERATE_DATE_ARRAY(
    '2024-01-01', '2024-01-07' -- dates
  ) AS week_dates;
```

Lab 3.6 – Querying JSON Data

Concepts

 **JSON in BigQuery** BigQuery has a native JSON data type that stores semi-structured data. Unlike STRING columns containing JSON text, the JSON type:

- Validates JSON at insert time
- Enables path-based extraction without parsing overhead
- Supports dot-notation access in some contexts

Create a Table with JSON

SQL

```
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.events` AS
SELECT * FROM UNNEST([
  STRUCT(
    'EVT-001' AS event_id,
    'page_view' AS event_type,
    TIMESTAMP '2024-04-01 10:00:00 UTC' AS event_time,
    PARSE_JSON('{"url":"/products/headphones","referrer":"google.com","session_id":"abc123","user":
{"id":"U001","country":"IN","is_logged_in":true},"device":{"type":"mobile","os":"Android","browser":"Chrome"}}') AS
event_data
  ),
  STRUCT(
    'EVT-002', 'add_to_cart', TIMESTAMP '2024-04-01 10:05:00 UTC',
    PARSE_JSON('{"product_id":"PROD-A1","price":299.99,"quantity":1,"cart_total":299.99,"session_id":"abc123"}')
  ),
  STRUCT(
    'EVT-003', 'purchase', TIMESTAMP '2024-04-01 10:15:00 UTC',
    PARSE_JSON('{"order_id":"ORD-009","items":[{"id":"PROD-A1","qty":1,"price":299.99},{id":"PROD-
F6","qty":2,"price":14.99}],"total":329.97,"payment":"credit_card"}')
  )
]);
```

JSON Extraction Functions

SQL

```
-- JSON_VALUE: extract a scalar value (returns STRING)
SELECT
  event_id,
  event_type,
  JSON_VALUE(event_data, '$.session_id') AS session_id,
  JSON_VALUE(event_data, '$.user.id') AS user_id,
  JSON_VALUE(event_data, '$.user.country') AS country,
  JSON_VALUE(event_data, '$.device.type') AS device_type,
  CAST(JSON_VALUE(event_data, '$.price') AS NUMERIC) AS price
FROM `bigquery-lab.ecommerce.events`
WHERE event_type = 'page_view';
```

SQL

```
-- JSON_QUERY: extract a JSON object or array (returns JSON string)
SELECT
  event_id,
  JSON_QUERY(event_data, '$.user') AS user_json, -- returns {"id":"U001",...}
  JSON_QUERY(event_data, '$.items') AS items_array -- returns the array
FROM `bigquery-lab.ecommerce.events`;
```

SQL

```
-- JSON_VALUE_ARRAY: extract a JSON array as a STRING array
SELECT
  event_id,
  JSON_VALUE_ARRAY(event_data, '$.items') AS items
FROM `bigquery-lab.ecommerce.events`
WHERE event_type = 'purchase';
```

SQL

```
-- Unnest JSON arrays using JSON_QUERY_ARRAY + UNNEST
SELECT
  e.event_id,
  JSON_VALUE(item, '$.id')           AS product_id,
  CAST(JSON_VALUE(item, '$.qty') AS INT64) AS quantity,
  CAST(JSON_VALUE(item, '$.price') AS NUMERIC) AS price
FROM `bigquery-lab.ecommerce.events` AS e,
     UNNEST(JSON_QUERY_ARRAY(e.event_data, '$.items')) AS item
WHERE e.event_type = 'purchase';
```

SQL

```
-- Check if a key exists in JSON
SELECT
  event_id,
  event_type,
  JSON_VALUE(event_data, '$.url') IS NOT NULL AS has_url,
  JSON_VALUE(event_data, '$.order_id') IS NOT NULL AS has_order
FROM `bigquery-lab.ecommerce.events`;
```

SQL

```
-- Build JSON using TO_JSON_STRING
SELECT
  order_id,
  TO_JSON_STRING(
    STRUCT(
      customer_id AS customer,
      amount      AS value,
      DATE(created_at) AS date
    )
  ) AS order_json
FROM `bigquery-lab.ecommerce.orders_raw`
LIMIT 5;
```

Lab 3.7 – Multi-Statement Queries

Concepts

 **Multi-Statement Queries** A multi-statement query (also called a script or procedure) contains two or more SQL statements separated by semicolons. BigQuery executes them in sequence within a single job. Supports variables, control flow, loops, and exception handling.

Requires billing account (not available in Sandbox due to DML restriction).

Variables

SQL

```
-- Declare and set variables
DECLARE start_date DATE DEFAULT '2024-01-01';
DECLARE end_date DATE;
DECLARE total_rev NUMERIC;

-- SET assigns a value
SET end_date = DATE_ADD(start_date, INTERVAL 3 MONTH);

-- SET with subquery result
SET total_rev = (
  SELECT SUM(amount)
  FROM `bigquery-lab.ecommerce.orders_raw`
  WHERE DATE(created_at) BETWEEN start_date AND end_date
  AND status = 'completed'
);

-- Use the variable in a query
SELECT
  FORMAT('Revenue from %t to %t: $%t', start_date, end_date, total_rev) AS summary;
```

IF / ELSE Control Flow

SQL

```
DECLARE revenue NUMERIC;

SET revenue = (SELECT SUM(amount) FROM `bigquery-lab.ecommerce.orders_raw`);

IF revenue > 1000 THEN
  SELECT 'High revenue month' AS result;
ELSEIF revenue > 500 THEN
  SELECT 'Medium revenue month' AS result;
ELSE
  SELECT 'Low revenue month - investigate' AS result;
END IF;
```

LOOP and WHILE

SQL

```
-- LOOP with EXIT WHEN (like a do-while)
DECLARE counter INT64 DEFAULT 0;
DECLARE results ARRAY<STRING> DEFAULT [];

LOOP
  SET counter = counter + 1;
  SET results = ARRAY_CONCAT(results, [FORMAT('Iteration %d', counter)]);
  EXIT WHEN counter >= 5;
END LOOP;

SELECT results;
```

SQL

```
-- FOR..IN loop over a query result
FOR row IN (
  SELECT category, COUNT(*) AS cnt
  FROM `bigquery-lab.ecommerce.orders_raw`
  GROUP BY category
) DO
  SELECT FORMAT('Category: %s → %d orders', row.category, row.cnt) AS log;
END FOR;
```

Exception Handling

SQL

```

DECLARE error_message STRING;

BEGIN
  -- This will succeed
  CREATE TEMP TABLE calc_results AS
  SELECT
    category,
    SUM(amount) AS revenue
  FROM `bigquery-lab.ecommerce.orders_raw`
  GROUP BY category;

  SELECT * FROM calc_results;

EXCEPTION WHEN ERROR THEN
  SET error_message = @@error.message;
  SELECT FORMAT('Script failed: %s', error_message) AS error;
END;

```

Practical Multi-Statement Script

SQL

```

-- End-to-end ETL script: extract, transform, load, report
DECLARE run_date DATE DEFAULT CURRENT_DATE();
DECLARE rows_inserted INT64;

-- Step 1: Create staging table
CREATE OR REPLACE TEMP TABLE staging AS
SELECT
  order_id,
  customer_id,
  UPPER(category) AS category,
  ROUND(amount, 2) AS amount,
  DATE(created_at) AS order_date,
  CASE
    WHEN amount >= 200 THEN 'large'
    WHEN amount >= 50 THEN 'medium'
    ELSE 'small'
  END AS order_size
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed';

-- Step 2: Check row count
SET rows_inserted = (SELECT COUNT(*) FROM staging);

-- Step 3: Only proceed if we have data
IF rows_inserted > 0 THEN
  -- Load to target
  CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.orders_transformed`
  AS SELECT * FROM staging;

  SELECT FORMAT('Success: inserted %d rows on %t', rows_inserted, run_date) AS status;
ELSE
  SELECT 'Warning: no rows found in source' AS status;
END IF;

```

Lab 3.8 – Recursive CTEs

Concepts

📖 **What are Recursive CTEs?** A recursive CTE uses `WITH RECURSIVE` to define a query that references itself. It repeatedly executes until a termination condition is met. Perfect for:

- Hierarchical data (org charts, category trees, folder structures)
- Graph traversal (shortest path, connected components)
- Generating sequences without a numbers table

Structure

```
WITH RECURSIVE cte_name AS (
  -- Anchor: initial rows (non-recursive part)
  SELECT ...

  UNION ALL

  -- Recursive: references cte_name itself
  -- Must converge - every iteration reduces the result set
  SELECT ...
  FROM cte_name
  WHERE <termination_condition>  -- REQUIRED to prevent infinite loop
)
SELECT * FROM cte_name;
```

SQL

Example 1 – Number Sequence

```
-- Generate numbers 1 through 10
WITH RECURSIVE numbers AS (
  SELECT 1 AS n          -- anchor

  UNION ALL

  SELECT n + 1           -- recursive step
  FROM numbers
  WHERE n < 10           -- termination: stop at 10
)
SELECT n FROM numbers ORDER BY n;
```

SQL

Example 2 – Date Series

```
-- Generate every date in Q1 2024 (Jan 1 to Mar 31)
WITH RECURSIVE date_series AS (
  SELECT DATE '2024-01-01' AS dt

  UNION ALL

  SELECT DATE_ADD(dt, INTERVAL 1 DAY)
  FROM date_series
  WHERE dt < DATE '2024-03-31'
)
SELECT dt FROM date_series ORDER BY dt;
```

SQL

Example 3 – Organizational Hierarchy

SQL

```

-- Create an org chart table
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.employees` AS
SELECT * FROM UNNEST([
  STRUCT(1 AS emp_id, 'Alice' AS name, NULL AS manager_id, 'CEO' AS role),
  STRUCT(2, 'Bob', 1, 'VP Engineering'),
  STRUCT(3, 'Carol', 1, 'VP Sales'),
  STRUCT(4, 'Dave', 2, 'Engineering Lead'),
  STRUCT(5, 'Eve', 2, 'Senior Engineer'),
  STRUCT(6, 'Frank', 4, 'Engineer'),
  STRUCT(7, 'Grace', 3, 'Sales Manager'),
  STRUCT(8, 'Henry', 7, 'Sales Rep')
]);

-- Find the full reporting chain for everyone under Alice
WITH RECURSIVE org_hierarchy AS (
  -- Anchor: start with Alice (CEO, no manager)
  SELECT
    emp_id,
    name,
    manager_id,
    role,
    0 AS depth,
    name AS path
  FROM `bigquery-lab.ecommerce.employees`
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: add direct reports of current level
  SELECT
    e.emp_id,
    e.name,
    e.manager_id,
    e.role,
    h.depth + 1,
    CONCAT(h.path, ' → ', e.name)
  FROM `bigquery-lab.ecommerce.employees` AS e
  JOIN org_hierarchy AS h ON e.manager_id = h.emp_id
)
SELECT
  emp_id,
  REPEAT(' ', depth) || name AS indented_name, -- visual indentation
  role,
  depth AS org_level,
  path AS reporting_chain
FROM org_hierarchy
ORDER BY path;

```


Example 4 – Category Tree Rollup

SQL

```
-- Create a product category hierarchy
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.categories` AS
SELECT * FROM UNNEST([
  STRUCT(1 AS cat_id, 'All Products' AS name, NULL AS parent_id),
  STRUCT(2, 'Electronics', 1),
  STRUCT(3, 'Clothing', 1),
  STRUCT(4, 'Home & Garden', 1),
  STRUCT(5, 'Audio', 2),
  STRUCT(6, 'Wearables', 2),
  STRUCT(7, 'Men', 3),
  STRUCT(8, 'Women', 3),
  STRUCT(9, 'Headphones', 5),
  STRUCT(10, 'Earbuds', 5)
]);

-- Find all ancestors of "Headphones" (cat_id = 9)
WITH RECURSIVE ancestors AS (
  SELECT cat_id, name, parent_id, 0 AS level
  FROM `bigquery-lab.ecommerce.categories`
  WHERE cat_id = 9

  UNION ALL

  SELECT c.cat_id, c.name, c.parent_id, a.level + 1
  FROM `bigquery-lab.ecommerce.categories` AS c
  JOIN ancestors AS a ON c.cat_id = a.parent_id
)
SELECT name, level
FROM ancestors
ORDER BY level;

-- Result: Headphones(0) → Audio(-1) → Electronics(-2) → All Products(-3)
```

Lab 3.9 – Creating and Running Saved Queries

Concepts

 **Saved Queries** Saved queries in BigQuery Studio let you store, share, and organize frequently-used SQL. They are stored in your project (or personal) and can be accessed from the Explorer panel. Unlike views, saved queries are not auto-executed — they're templates you run on demand.

Step 1 – Save a Query

1. Write a query in the editor
2. Click the **Save** dropdown → **Save query**
3. Give it a name: **Monthly Revenue Summary**
4. Choose **Project queries** (shareable) or **My queries** (personal)
5. Click **Save**

Step 2 — Organize Saved Queries

```
-- Example: a parameterized saved query using session variables
-- Save this as "Orders by Date Range"
DECLARE start_dt DATE DEFAULT DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY);
DECLARE end_dt   DATE DEFAULT CURRENT_DATE();

SELECT
  DATE(created_at) AS order_date,
  COUNT(*)        AS orders,
  SUM(amount)     AS revenue
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE DATE(created_at) BETWEEN start_dt AND end_dt
  AND status = 'completed'
GROUP BY order_date
ORDER BY order_date;
```

SQL

Step 3 — Find Saved Queries via API

```
# List saved queries using bq CLI (stored as routines or in job history)
bq ls --saved_queries

# Run a saved query
bq query --use_saved_query=<query_id>
```

SHELL

Step 4 — Schedule Queries (Requires Billing)

```
-- Scheduled queries run automatically on a cron schedule
-- Set up via: BigQuery Studio → Scheduled Queries → Create

-- Example query to schedule (runs daily, writes to a report table)
CREATE OR REPLACE TABLE `bigquery-lab.ecommerce.daily_revenue_report`
AS
SELECT
  DATE(created_at) AS report_date,
  category,
  COUNT(*)        AS orders,
  SUM(amount)     AS revenue,
  CURRENT_TIMESTAMP() AS generated_at
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE DATE(created_at) = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY)
GROUP BY report_date, category;
```

SQL

Lab 3.10 — Optimize Queries

Concepts

📖 Query Optimization BigQuery charges based on bytes scanned. Optimization is both a performance and cost concern. The key principle: read only what you need.

Optimization Strategy 1 — Select Only Needed Columns

```
-- BAD: scans ALL columns (expensive on wide tables)
SELECT * FROM `bigquery-lab.ecommerce.orders_raw`;

-- GOOD: only scans 3 columns
SELECT order_id, amount, status
FROM `bigquery-lab.ecommerce.orders_raw`;
```

SQL

Optimization Strategy 2 – Use Partition Filters

SQL

```
-- BAD: scans entire table across all partitions
SELECT COUNT(*) FROM `bigquery-lab.ecommerce.orders_partitioned`;

-- GOOD: only scans Jan 2024 partition
SELECT COUNT(*)
FROM `bigquery-lab.ecommerce.orders_partitioned`
WHERE DATE(created_at) >= '2024-01-01'
AND DATE(created_at) < '2024-02-01';
```

Optimization Strategy 3 – Use Clustering Columns in Filters

SQL

```
-- GOOD: cluster filter prunes storage blocks
SELECT order_id, amount
FROM `bigquery-lab.ecommerce.orders_clustered`
WHERE category = 'Electronics' -- first cluster column
AND status = 'completed'; -- second cluster column
```

Optimization Strategy 4 – TABLESAMPLE

SQL

```
-- Sample 10% of table rows for exploratory analysis
-- Reduces bytes scanned by ~90%
SELECT
  category,
  APPROX_COUNT_DISTINCT(customer_id) AS approx_customers,
  AVG(amount) AS avg_order
FROM `bigquery-lab.ecommerce.orders_clustered`
TABLESAMPLE SYSTEM (10 PERCENT) -- 10% random sample
GROUP BY category;
```

Optimization Strategy 5 – Avoid Self-Joins with Window Functions

SQL

```
-- BAD: self-join to compute running total
SELECT
  a.order_id,
  a.amount,
  SUM(b.amount) AS running_total
FROM `bigquery-lab.ecommerce.orders_raw` AS a
JOIN `bigquery-lab.ecommerce.orders_raw` AS b
  ON b.created_at <= a.created_at
GROUP BY a.order_id, a.amount;

-- GOOD: window function (single scan)
SELECT
  order_id,
  amount,
  SUM(amount) OVER (ORDER BY created_at ROWS UNBOUNDED PRECEDING) AS running_total
FROM `bigquery-lab.ecommerce.orders_raw`;
```

Optimization Strategy 6 – Set Maximum Bytes Billed

SQL

```
-- Embed a safety limit directly in your query (requires DDL in a script)
-- In the Console: Query Settings → Maximum bytes billed = 1073741824 (1 GB)

-- In bq CLI:
-- bq query --maximum_bytes_billed=1073741824 "SELECT ..."
```

Optimization Strategy 7 – APPROX Functions

```
-- APPROX_COUNT_DISTINCT is ~97% accurate but scans far less data
-- than COUNT(DISTINCT ...) on large tables
SELECT
  category,
  APPROX_COUNT_DISTINCT(customer_id) AS approx_unique_customers,
  COUNT(DISTINCT customer_id)      AS exact_unique_customers  -- expensive on big tables
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY category;
```

SQL

Optimization Strategy 8 – Use WHERE Before JOIN

```
-- GOOD: filter in subquery before joining (BQ optimizer usually does this, but be explicit)
SELECT o.order_id, c.first_name
FROM (
  SELECT order_id, customer_id
  FROM `bigquery-lab.ecommerce.orders_raw`
  WHERE status = 'completed'      -- filter here, not after join
) AS o
JOIN `bigquery-lab.ecommerce.customers` AS c USING (customer_id);
```

SQL

Inspect Query Execution Plan

```
# Get detailed job stats including bytes processed per stage
bq show --format=prettyjson -j <job_id>

# Or in the Console: after running a query, click "Execution details"
```

SHELL

Lab 3.11 – Query External Tables

This topic was covered in [Lab 1.5 — Create and Query External Tables](#). Here we extend with advanced patterns.

External Table with Schema Definition

```
-- Create external table with explicit schema from multiple CSV files
-- The '*' wildcard matches all CSV files in the folder
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.sales_history_ext`
(
  year      INT64,
  month     INT64,
  category  STRING,
  revenue   NUMERIC,
  order_count INT64
)
OPTIONS (
  format          = 'CSV',
  uris            = ['gs://YOUR_BUCKET/sales/year=*/month=*/*.csv'],
  skip_leading_rows = 1,
  null_marker     = 'NULL',
  quote           = '',
  field_delimiter = ','
);
```

SQL

Hive-Partitioned External Table

SQL

```
-- When GCS files are organized as Hive partitions (e.g., dt=2024-01-01/)
-- BigQuery can use the folder structure for partition pruning
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.logs_partitioned_ext`
OPTIONS (
  format      = 'PARQUET',
  uris        = ['gs://YOUR_BUCKET/logs/*'],
  hive_partition_uri_prefix = 'gs://YOUR_BUCKET/logs',
  require_hive_partition_filter = FALSE
);
-- Files expected at: gs://YOUR_BUCKET/logs/dt=2024-01-01/data.parquet
-- Then filter with: WHERE dt = '2024-01-01'
```

BigLake Table (Enhanced External Table)

SQL

```
-- BigLake tables provide row/column-level security on external data
-- Requires billing account
CREATE OR REPLACE EXTERNAL TABLE `bigquery-lab.ecommerce.sensitive_ext`
WITH CONNECTION `us.my-cloud-resource-connection`
OPTIONS (
  format = 'PARQUET',
  uris   = ['gs://YOUR_BUCKET/sensitive/*.parquet']
);
```

Lab 3.12 – Logical Views

Concepts

 **What is a Logical View?** A logical view (or just "view") is a saved SQL query stored as a named object. When you query a view, BigQuery executes the underlying SQL in real time — no data is stored. Views are useful for:

- Abstracting complex queries into reusable objects
- Applying security (column masking, row filtering)
- Providing a stable schema over changing underlying tables

Feature	View	Materialized View
Data stored	No (query runs each time)	Yes (cached results)
Always current	Yes	Periodically refreshed
Query cost	Full scan each time	Cheaper (pre-aggregated)
Supports DML	No	No
Nesting limit	Up to 16 levels	Cannot reference other MVs

Create Views

SQL

```
-- Simple view: completed orders with customer info
CREATE OR REPLACE VIEW `bigquery-lab.ecommerce.v_completed_orders` AS
SELECT
  o.order_id,
  o.customer_id,
  c.first_name,
  c.last_name,
  c.country,
  o.amount,
  o.category,
  DATE(o.created_at) AS order_date
FROM `bigquery-lab.ecommerce.orders_raw` AS o
LEFT JOIN `bigquery-lab.ecommerce.customers` AS c
  USING (customer_id)
WHERE o.status = 'completed';
```

SQL

```
-- Analytical view: customer lifetime value with tier
CREATE OR REPLACE VIEW `bigquery-lab.ecommerce.v_customer_ltv` AS
WITH orders_agg AS (
  SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(amount) AS lifetime_value,
    MIN(DATE(created_at)) AS first_order_date,
    MAX(DATE(created_at)) AS last_order_date
  FROM `bigquery-lab.ecommerce.orders_raw`
  WHERE status = 'completed'
  GROUP BY customer_id
)
SELECT
  a.*,
  CASE
    WHEN a.lifetime_value >= 500 THEN 'Platinum'
    WHEN a.lifetime_value >= 200 THEN 'Gold'
    WHEN a.lifetime_value >= 50 THEN 'Silver'
    ELSE 'Bronze'
  END AS tier,
  DATE_DIFF(a.last_order_date, a.first_order_date, DAY) AS days_since_first_order
FROM orders_agg AS a;
```

SQL

```
-- Security view: masks email address (PII protection)
-- Shows only domain portion of email
CREATE OR REPLACE VIEW `bigquery-lab.ecommerce.v_customers_safe` AS
SELECT
  customer_id,
  first_name,
  last_name,
  CONCAT('***@', SPLIT(email, '@')[OFFSET(1)]) AS masked_email,
  country,
  signup_date,
  is_premium
FROM `bigquery-lab.ecommerce.customers`;
```

Query Views

```
-- Views are queried exactly like tables
SELECT
  country,
  tier,
  COUNT(*) AS customers,
  AVG(lifetime_value) AS avg_ltv
FROM `bigquery-lab.ecommerce.v_customer_ltv` -- this runs the full underlying query
GROUP BY country, tier
ORDER BY avg_ltv DESC;
```

SQL

Inspect Views

```
-- List all views in the dataset
SELECT
  table_name,
  view_definition
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.VIEWS
ORDER BY table_name;
```

SQL

```
-- Check what columns a view exposes
SELECT column_name, data_type
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'v_completed_orders'
ORDER BY ordinal_position;
```

SQL

Lab 3.13 — Materialized Views

Concepts

📖 **What is a Materialized View?** A materialized view (MV) pre-computes and caches the results of a query. Unlike a logical view, the results are physically stored in BigQuery. When the underlying base table changes, BigQuery incrementally updates the MV — you don't have to refresh it manually.

BigQuery also performs **smart rewriting**: if a query against a base table can be resolved by an MV, BigQuery automatically routes it through the MV — even if you didn't mention the MV in your query!

Materialized View Constraints

- ✅ Supported: SELECT, FROM (single table), WHERE, GROUP BY, aggregate functions (SUM, COUNT, AVG, MIN, MAX, COUNTIF, HLL_COUNT.INIT)
- ❌ Not supported: JOINS, UNION, window functions, subqueries, LIMIT, ORDER BY
- ❌ Cannot: reference other MVs, query external tables, reference logical views

Create Materialized Views

```
-- Aggregate MV: pre-compute daily revenue by category
CREATE MATERIALIZED VIEW IF NOT EXISTS `bigquery-lab.ecommerce.mv_daily_revenue`
PARTITION BY order_date    -- MV can be partitioned too!
CLUSTER BY category
OPTIONS (
  enable_refresh           = TRUE,  -- auto-refresh when base table changes
  refresh_interval_minutes = 60,    -- check for changes every 60 minutes
  description              = 'Pre-aggregated daily revenue by category'
)
AS
SELECT
  DATE(created_at) AS order_date,
  category,
  COUNT(*)         AS order_count,
  SUM(amount)      AS total_revenue,
  AVG(amount)      AS avg_order_value,
  COUNT(DISTINCT customer_id) AS unique_customers
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed'
GROUP BY order_date, category;
```

SQL

```
-- Simpler MV: product popularity
CREATE MATERIALIZED VIEW IF NOT EXISTS `bigquery-lab.ecommerce.mv_product_stats`
AS
SELECT
  product_id,
  COUNT(*)    AS times_ordered,
  SUM(amount) AS total_revenue
FROM `bigquery-lab.ecommerce.orders_raw`
GROUP BY product_id;
```

SQL

Query Materialized Views

```
-- Direct query (uses stored cache – fast and cheap)
SELECT *
FROM `bigquery-lab.ecommerce.mv_daily_revenue`
WHERE order_date >= '2024-01-01'
ORDER BY order_date, total_revenue DESC;
```

SQL

```
-- Smart rewriting in action:
-- This query hits orders_raw, but BigQuery rewrites it to use mv_daily_revenue
-- No MV reference needed – BigQuery does it automatically!
SELECT
  DATE(created_at) AS order_date,
  category,
  SUM(amount) AS revenue
FROM `bigquery-lab.ecommerce.orders_raw`
WHERE status = 'completed'
GROUP BY 1, 2;
-- Check "Execution details" tab – you'll see mv_daily_revenue used
```

SQL

Manage Materialized Views

```
-- Manually trigger a refresh (instead of waiting for auto-refresh)
CALL BQ.REFRESH_MATERIALIZED_VIEW(`bigquery-lab.ecommerce.mv_daily_revenue`);
```

SQL

SQL

```
-- Check MV metadata
SELECT
  table_name,
  last_refresh_time,
  refresh_watermark
FROM `bigquery-lab.ecommerce`.INFORMATION_SCHEMA.MATERIALIZED_VIEWS
ORDER BY table_name;
```

SQL

```
-- Alter MV options (e.g., change refresh interval)
ALTER MATERIALIZED VIEW `bigquery-lab.ecommerce.mv_daily_revenue`
SET OPTIONS (
  refresh_interval_minutes = 30
);
```

SQL

```
-- Drop a materialized view
DROP MATERIALIZED VIEW IF EXISTS `bigquery-lab.ecommerce.mv_product_stats`;
```

Capstone Exercise

Combine everything you've learned in one end-to-end analysis:

```

-- CAPSTONE: Full customer analytics pipeline
WITH

-- Step 1: Flatten sessions with unnested page views
session_pages AS (
  SELECT
    session_id,
    customer_id,
    page_view,
    ROW_NUMBER() OVER (PARTITION BY session_id ORDER BY pos) AS step
  FROM `bigquery-lab.ecommerce.user_sessions`,
       UNNEST(page_views) AS page_view WITH OFFSET AS pos
),

-- Step 2: Identify sessions that converted (reached checkout)
converted_sessions AS (
  SELECT DISTINCT session_id
  FROM session_pages
  WHERE page_view = 'checkout'
),

-- Step 3: Funnel metrics per session
session_funnel AS (
  SELECT
    s.session_id,
    s.customer_id,
    MAX(s.step) AS total_steps,
    COUNTIF(s.page_view = 'cart') > 0 AS reached_cart,
    COUNTIF(s.page_view = 'checkout') > 0 AS converted,
    s.session_id IN (SELECT session_id FROM converted_sessions) AS is_converted
  FROM session_pages AS s
  GROUP BY s.session_id, s.customer_id
),

-- Step 4: Join with order revenue using recursive customer tier
customer_orders AS (
  SELECT
    customer_id,
    SUM(amount) AS lifetime_value,
    COUNT(*) AS orders
  FROM `bigquery-lab.ecommerce.orders_raw`
  WHERE status = 'completed'
  GROUP BY customer_id
),

-- Final: Combined customer profile
SELECT
  f.customer_id,
  COUNT(f.session_id) AS total_sessions,
  COUNTIF(f.is_converted) AS converted_sessions,
  ROUND(
    SAFE_DIVIDE(COUNTIF(f.is_converted), COUNT(f.session_id)) * 100, 1
  ) AS conversion_rate_pct,
  COALESCE(o.lifetime_value, 0) AS lifetime_value,
  COALESCE(o.orders, 0) AS completed_orders,
  CASE
    WHEN COALESCE(o.lifetime_value, 0) >= 500 THEN '💎 Platinum'
    WHEN COALESCE(o.lifetime_value, 0) >= 200 THEN '🥇 Gold'
    WHEN COALESCE(o.lifetime_value, 0) >= 50 THEN '🥈 Silver'
    ELSE '🥉 Bronze'
  END AS tier
FROM session_funnel AS f

```

```
LEFT JOIN customer_orders AS o USING (customer_id)
GROUP BY f.customer_id, o.lifetime_value, o.orders
ORDER BY lifetime_value DESC;
```

Quick Reference

Free Tier Limits

Resource	Limit	Notes
Query processing	1 TB/month	Resets 1st of each month
Storage	10 GB/month	Active storage
Table retention (Sandbox)	60 days	Add billing for permanent
DML (INSERT/UPDATE/DELETE)	✗ Sandbox	✓ Billing account (free tier)
Streaming inserts	✗ Sandbox	Requires billing
Materialized views	✓ With billing	60-day expiry in Sandbox

Essential bq CLI Commands

SHELL

```
# Datasets
bq mk --dataset PROJECT:DATASET
bq ls PROJECT:
bq show PROJECT:DATASET
bq rm -r PROJECT:DATASET

# Tables
bq show PROJECT:DATASET.TABLE
bq head -n 10 PROJECT:DATASET.TABLE
bq rm PROJECT:DATASET.TABLE

# Queries
bq query --use_legacy_sql=false "YOUR SQL"
bq query --dry_run "YOUR SQL"
bq query --maximum_bytes_billed=1073741824 "YOUR SQL"

# Load data
bq load --autodetect --source_format=CSV \
  PROJECT:DATASET.TABLE gs://BUCKET/file.csv

# Export data
bq extract PROJECT:DATASET.TABLE gs://BUCKET/export_*.csv
```

Common INFORMATION_SCHEMA Queries

SQL

```
-- All datasets
SELECT * FROM `PROJECT`.INFORMATION_SCHEMA.SCHEMATA;

-- All tables in a dataset
SELECT * FROM `PROJECT.DATASET`.INFORMATION_SCHEMA.TABLES;

-- All columns
SELECT * FROM `PROJECT.DATASET`.INFORMATION_SCHEMA.COLUMNS;

-- All views
SELECT * FROM `PROJECT.DATASET`.INFORMATION_SCHEMA.VIEWS;

-- All materialized views
SELECT * FROM `PROJECT.DATASET`.INFORMATION_SCHEMA.MATERIALIZED_VIEWS;

-- Partitions
SELECT * FROM `PROJECT.DATASET`.INFORMATION_SCHEMA.PARTITIONS;

-- Recent job history
SELECT
  job_id, creation_time, total_bytes_processed, total_slot_ms, state
FROM `PROJECT`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE DATE(creation_time) = CURRENT_DATE()
ORDER BY creation_time DESC
LIMIT 20;
```